

dispatch_ffn_combine_kernel 算子解析报告

本文面向已经熟悉 CUDA/GPU，但不熟悉 Ascend C / vllm-ascend MC2 算子的读者，重点解释 `dispatch_ffn_combine_kernel` 的算子级执行流程。核心代码位于：

- `op_kernel/dispatch_ffn_combine_kernel.hpp`
- `op_kernel/dispatch_ffn_combine.h`
- `op_kernel/dispatch_ffn_combine.cpp`
- `op_host/dispatch_ffn_combine_tiling.cpp`
- `op_kernel/utils/block_epilogue_pertoken_v2.hpp`
- `op_kernel/utils/block_epilogue_pertoken_row.hpp`
- `op_kernel/utils/block_epilogue_pertoken_swiglu.hpp`
- `op_kernel/utils/block_mmad_preload_async_fixpipe_quant.hpp`
- `op_kernel/utils/hccl_shmem.hpp`

1. 算子整体作用

`dispatch_ffn_combine_kernel` 是一个面向 MoE Expert Parallel 场景的融合算子。它把 MoE 推理或训练中的三个典型阶段放进同一个 Ascend C kernel：

- **Dispatch**：根据 `expert_idx / expertId` 把输入 token 路由到对应 expert。跨 rank 的 Expert Parallel 场景下，一个 token 可能需要发给远端 rank 上的 expert。本实现先做本地 routing / sorting / dynamic quant，然后通过 HCCL window / peermem 访问完成 rank 间 token 搬运。
- **FFN**：每个 expert 对接收到的 token 执行两层 FFN。这里主要是 $x * w1 \rightarrow \text{SwiGLU} \rightarrow * w2$ 。AIC/Cube 负责两次 GEMM：`GMM1` 和 `GMM2`，AIV/Vector 负责 SwiGLU、量化/反量化和搬运。
- **Combine**：把各 expert 计算后的 token 结果按照原始 token/topK 关系合并回去。跨 rank 场景下，结果需要写回目标 rank 的 peermem，再由本 rank unpermute 到最终输出 `out`。

该 kernel 把 dispatch、ffn、combine 融合在一起，主要是为了减少传统分阶段 MoE pipeline 的额外开销：

- 减少中间 tensor 落 GM 的次数，例如 dispatch 后的 permuted token、FFN 中间结果、combine 前结果不必在多个独立算子之间反复写出/读入。
- 减少多 kernel launch 和 host 调度开销。
- 减少独立 all-to-all / reorder / GEMM / combine 算子之间的全局同步。
- 让 AIV 的通信搬运、routing、epilogue 与 AIC 的 GEMM 形成生产者-消费者流水。
- 利用 workspace 和 peermem 的固定布局复用中间 buffer，避免普通 pipeline 中多份临时 buffer。

从代码看，这个算子是一个 **MC2 通信 + 计算融合算子**：`dispatch_ffn_combine_def.cpp` 中为算子注册了 `.MC2().HcclGroup("group")`，host tiling 中配置了 `AlltoAll=level0:fullmesh;level1:pairwise`，device 端通过 `HcclShmem` 访问本 rank 和远端 rank 的 window。

2. 输入输出与核心数据结构

2.1 顶层输入输出

PyTorch 适配层 `dispatch_ffn_combine_torch_adpt.h` 第 20-83 行暴露了：

- `x`：输入 token 矩阵，shape 推断为 `[M, K]`。
- `weight1`：第一层 expert 权重列表，对应 FFN 第一层，代码中为 `w1 / ptrB1`。
- `weight2`：第二层 expert 权重列表，对应 FFN 第二层，代码中为 `w2 / ptrB2`。
- `expert_idx`：每个 token 的 topK expert 路由结果，shape 推断为 `[M, topK]`。
- `scale1 / scale2`：量化权重或 GEMM fixpipe 相关 scale，代码中类型为 `uint64_t` scale GM 指针。
- `probs`：topK gating 权重，用于最终 unpermute / weighted combine。
- `group`：HCCL 通信域名称。
- `max_output_size`：每 rank / 每轮最多处理的 routed token 上限。
- `x_active_mask`：可选 bool mask。若 token inactive，`ApplyXActiveMask` 会把对应 expert id 置为 sentinel。
- `out`：最终输出，dtype 与输入/配置相关。
- `expert_token_nums`：每个本地 expert 收到的 token 数。

算子定义见 `op_host/dispatch_ffn_combine_def.cpp` 第 21-74 行：输入包括 `a`、动态 `w1`、动态 `w2`、`expertIdx`、动态 `scale1`、动态 `scale2`、`probs`、可选 `xActiveMaskOptional`；输出为 `out` 和 `expert_token_nums`。

kernel 入口在 `op_kernel/dispatch_ffn_combine.cpp` 第 22-32 行：

- 入参：`x, w1, w2, expertId, scale1, scale2, probs, xActiveMask, c, expertTokenNums, workspaceGM, tilingGM`
- 当 `TILING_KEY_IS(1000010)` 时，使用 `KERNEL_TYPE_MIX_AIC_1_2`，实例化 `DispatchFFNCombine<int8_t, DTYPE_W1, DTYPE_OUT, false, true>`。

2.2 Params

`DispatchFFNCombineKernel::Params` 定义在 `op_kernel/dispatch_ffn_combine_kernel.hpp` 第 99-185 行，是 device 侧主参数结构。

核心字段包括：

- `problemShape`：`GemmCoord{M, N, K}`，在 `dispatch_ffn_combine.h` 第 262 行构造。

- `ptrA`: 原始输入 `x` 的 GM 地址。
- `ptrB1 / ptrB2`: 两层 expert 权重地址。
- `ptrScale1 / ptrScale2`: 两层 GEMM scale 地址。
- `ptrOutput`: 最终输出 `out`。
- `ptrWorkspace`: 融合算子 workspace。
- `ptrExpertTokenNums`: 输出 `expert_token_nums`。
- `EP`: Expert Parallel world size, 通常等于 rank 数。
- `rank / rankSize`: 本 rank id 和通信域大小。
- `expertPerRank`: 每个 rank 持有的 expert 数。
- `listLen`: 动态权重 list 长度。若 `listLen == 1`, 代码认为所有 expert 权重在一个 tensor 内; 否则按 list 索引取权重。
- `maxOutputSize`: 当前 rank 本轮最多承载的 dispatched token 数。
- `topK`: 每个 token 路由到的 expert 数。
- `expertIdx`: 路由矩阵地址。
- `probs`: gating 权重地址。
- `ptrXActiveMask`: 可选 active mask。
- `initRoutingQuantTilingKey / moeInitRoutingQuantV2TilingData`: InitRouting + dynamic quant 子流程 tiling。
- `epilogueCoreNum / epilogueGranularity`: AIV epilogue 分核与 GMM1/SwiGLU 流水切分粒度。

这些参数在 `DispatchFFNCombine::Process` 中构造, 见 `op_kernel/dispatch_ffn_combine.h` 第 269-282 行。

2.3 Host tiling 参数

`op_kernel/dispatch_ffn_combine_tiling.h` 第 21-56 行定义:

- `DispatchFFNCombineInfo`: `M/K/N/expertPerRank/maxOutputSize/aivNum/topK/worldSize/listLen` 等。
- `CoCTiling`: `m0/k0/n0/swizzleDirect/swizzleOffset/ubMoveNum/commNpuSplit/commDataSplit/lenPerLoop`, 以及 InitRouting quant tiling。
- `DispatchFFNCombineTilingData`: 包含 `Mc2InitTiling`、`Mc2CcTiling`、`DispatchFFNCombineInfo`、`CoCTiling`。

Host tiling 逻辑在 `op_host/dispatch_ffn_combine_tiling.cpp`:

- 第 86-115 行解析 `group`、`maxOutputSize`、`transB`、`weightNz`, 并通过 `HcomTopoInfo::GetGroupRankSize` 得到 `worldSize`。
- 第 119-159 行从 `x`、`w1`、`expertIdx` 推导 `M/K/N/topK/listLen/expertPerRank`。
- 第 237-251 行设置 `blockDim` 和 `tilingKey`。
- 第 255-279 行构造 `MoeInitRoutingQuantV2TilingBase`, 把 routing/quant tiling 塞进 `cocTiling`。
- 第 281-287 行检查 HCCL window size 是否足够。
- 第 297-307 行推导 workspace 大小。
- 第 310-317 行配置 MC2 通信: `AlltoAll=level0:fullmesh;level1:pairwise`。

2.4 WorkspaceInfo

`WorkspaceInfo` 在 `dispatch_ffn_combine_kernel.hpp` 第 1112-1168 行。它把 `ptrWorkspace` 线性切成多个中间 buffer:

- `expandedRowIndex`: workspace 起始位置。InitRouting 产生的 expanded/sorted row index, 最终 unpermute 用。
- `ptrCumsumMM`: `cumsumMM`, 按 rank/expert 统计的 token 前缀和, 用来确定每个 expert group 的 GEMM M 维大小和行偏移。
- 第二段 `EP * EP * expertPerRank * sizeof(int32_t)` 被跳过但没有命名字段。结合 host workspace 三倍计数空间推断, 这里可能预留给 InitRouting / token count 相关中间区, 需要进一步确认。
- `ptrPerTokenScale`: 第一阶段 per-token dynamic quant scale, 供 GMM1 输出 SwiGLU 前反量化使用。
- `ptrPerTokenScale2`: SwiGLU 后重新量化得到的 per-token scale, 供 GMM2 输出 combine 反量化使用。
- `ptrTokenPerExpert`: workspace 内 token count 区; 但实际 kernel 里 `tokenPerExpert` 绑定到 peermem 的 `offsetPeerTokenPerExpert`, 此字段未见直接使用, 需要进一步确认。
- `ptrC`: GMM1 输出 workspace, shape 近似 `[maxOutputSize, N]`。
- `ptrC2`: GMM2 输出 workspace, shape 近似 `[maxOutputSize, K]`。
- `ptrA`: 本 rank 聚合后的 dispatched token buffer, AIC 的 GMM1 输入。
- `ptrPermutedToken`: SwiGLU 后、GMM2 前的 int8 中间结果, shape 近似 `[maxOutputSize, N/2]`。
- `ptrSumBeforeRank`: `preSumBeforeRank`, combine 写回远端 peermem 时的目标行偏移。
- `ptrSoftFlagBase`: soft flag 区。代码中定义了 `InitArithProgress / UpdateAicFlags`, 但当前主路径没有调用, 是否为遗留或未启用路径需要进一步确认。

Host workspace 估算在 `dispatch_ffn_combine_tiling.cpp` 第 297-307 行, 与上述 buffer 基本对应。

2.5 PeermemInfo

`PeermemInfo` 在 `dispatch_ffn_combine_kernel.hpp` 第 1170-1185 行, 把 HCCL window / peermem 切为:

- `offsetA = 0`: 远端 rank 写入或本 rank 读取的 routed + quantized token。
- `offsetPeerPerTokenScale = offsetA + AlignUp(shmem.SegmentSize() / 3, 512)`: peermem 中每 token scale 区。
- `offsetD = offsetPeerPerTokenScale + MB_SIZE`: combine 后结果区。
- `offsetPeerTokenPerExpert = shmem.SegmentSize() - 2 * MB_SIZE`: tokenPerExpert 统计区。

`HcclShmem` 在 `op_kernel/utils/hccl_shmem.hpp` 第 84-183 行封装本地/远端 window 访问:

- `shmem()`: 本 rank local window。
- `shmem(rankId)`: 远端 rank base。
- `shmem(offset, rankId)`: 远端 rank 某 offset 地址。
- `CrossRankSync()`: 全 rank 栅栏。

2.6 token、expert、rank、group、block、core 的关系

- **token**: 原始输入 x 的一行。每个 token 有 `topK` 个 expert 目标。
- **expert**: 被切分到不同 rank。总 expert 数约为 `EP * expertPerRank`。
- **rank / EP**: Expert Parallel 通信域中的设备编号。`EP` 在 device 侧表示 world size。
- **groupIdx**: 本 rank 内 expert 编号, 范围 `[0, expertPerRank)`。AIC 的 GMM1/GMM2 都按 `groupIdx` 分组计算。
- **dstEpIdx**: 代码中经常表示目标/远端 rank 编号, 范围 `[0, EP)`。
- **block**: GEMM tile 调度单位。`L1TileShape = GemmShape<128, 256, 512>`, 即 GEMM 逻辑 tile 的 M/N/K 分块。
- **AIC core**: 执行 Cube/GEMM。
- **AIV core / subblock**: 执行 routing、copy、scale、SwiGLU、combine、unpermute 等 vector / MTE 操作。AIV 侧 `coreIdx = get_block_idx() + get_subblockid() * get_block_num()`, 见 `dispatch_ffn_combine_kernel.hpp` 第 196-198 行。

2.7 关键变量语义

- `expandedRowIdx`: InitRouting 生成的 expanded token 到原 token/topK 关系的索引。最后 `KernelMoeTokenUnpermute` 用它从 `peermem offsetD` 读回结果并按 `probs` 加权写 `out`。见第 806-811 行和第 998-1002 行。
- `perTokenScale / gmPerTokenScale1`: dynamic quant 后每个 dispatched token 的量化 scale。GMM1 输出 `gmC` 在 `BlockEpilogue1` 中乘它做反量化, 见 `block_epilogue_pertoken_swiglu.hpp` 第 213-220 行。
- `gmPerTokenScale2`: SwiGLU 后再量化产生的 scale。GMM2 输出 `gmC2` 在 `CombineV1/V2` 中乘它反量化, 见 `block_epilogue_pertoken_v2.hpp` 第 159-179 行。
- `tokenPerExpert`: 三维逻辑布局, `tokenPerExpertLayout(dstEpIdx, rank, groupIdx)` 表示“来自/面向某 rank 与本 rank、某 expert 组之间”的 token 数。具体 `dim0/dim1` 含义在不同阶段的读写方向中容易混淆; 结合 `Pull` 和 `Combine` 代码推断, 它保存每个 rank 对每个目标 rank、每个 expert 的 token 数矩阵。
- `cumsumMM`: 按 rank 维对 `tokenPerExpert` 做前缀和, `cumsumMM((EP - 1) * expertPerRank + groupIdx)` 是本 rank 的某 expert group 收到的总 token 数, 供 GMM 的 M 维使用。
- `preSumBeforeRank / sumBeforeRank`: combine 写回目标 rank `peermem` 时的目标行偏移。`preSumBeforeRank(dstEpIdx * expertPerRank + groupIdx)` 表示当前 rank 写回 `dstEpIdx` 的该 expert 组结果时, 在目标 `peermem` 中应该写到该 expert 分段内的起始行。该语义来自 `CrossRankSyncAndLocalTokenPerExpertAllGatherAndGetSumPreRankV2` 第 710-724 行和 `CombineV1/V2` 使用处。
- `preSrcExpertSum`: `CombineV2` 中当前 expert group 在本地 `gmC2` 中的起始行累计偏移, 见第 1057、1105 行。
- `prevGroupSum1 / prevGroupSum2`: Dispatch Pull / `CombineV1` 中前面 expert group 的累计计数, 用于拼接 workspace 中按 expert group 排列的 token。
- `epilogueGranularity`: 把 GMM1/SwiGLU 切成两波的 group 粒度, 在 `dispatch_ffn_combine.h` 第 264-268 行设置为 `expertPerRank - 3` 或小于 expert 数时 `expertPerRank - 1`。

AIC 主要消费:

- `gmA`: AIV 从 `peermem pull` 到本地 workspace 的 dispatched token。
- `ptrB1/ptrB2`: expert 权重。
- `ptrScale1/ptrScale2`: GEMM scale。
- `gmPermutedToken`: SwiGLU 后、GMM2 前的 int8 中间结果。

AIV 主要消费/产生:

- 消费 `x`、`expertIdx`、`probs`、`xActiveMask`。
- 产生 `expandedRowIdx`、`tokenPerExpert`、`cumsumMM`、`preSumBeforeRank`。
- 搬运 `peermem offsetA` 到 workspace `gmA`。
- 对 `gmC` 做 SwiGLU 和量化, 产生 `gmPermutedToken`、`gmPerTokenScale2`。
- 对 `gmC2` 做反量化并 scatter 到远端 `peermem offsetD`。
- 最后 unpermute 到 `out`。

3. 顶层执行流程

阶段 0: Host / kernel 入口与关键初始化

目标: 根据 tiling 选择 kernel 类型, 构造 device 侧类型和参数。

参与者: Host tiling、kernel 入口、AIC/AIV 混合任务。

关键代码:

- `op_kernel/dispatch_ffn_combine.cpp` 第 22-32 行。
- `op_kernel/dispatch_ffn_combine.h` 第 170-286 行。
- `dispatch_ffn_combine_kernel.hpp` 第 189-253 行。

输入: `x/w1/w2/expertId/scale1/scale2/probs/xActiveMask/out/expertTokenNums/workspaceGM/tilingGM`。

输出: 构造 `DispatchFFNCombineKernel::Params`, 初始化 workspace tensor 与 `peermem` 布局。

说明：

`DispatchFFNCombine::Process` 选择 `Arch::AtlasA2`、`BlockMmad`、三个 `BlockEpilogue` 和 `BlockScheduler`。`DispatchFFNCombineKernel` 构造时区分 AIC/AIV 获取 `coreIdx/coreNum`，并在 `initBuffer` 中设置 `gmA/gmC/gmC2/gmPermutedToken/gmPerTokenScale1/gmPerTokenScale2/tokenPerExpert/cumsumMM/preSumBeforeRank`。

`operator()<AIC>` 调用 `GMM1`，等待 `SYNCFLAGV2C`，再调用 `GMM2`；`operator()<AIV>` 调用 `DispatchAndCombine`。见 `dispatch_ffn_combine_kernel.hpp` 第 213-228 行。

阶段 1: ApplyXActiveMask

目标：如果提供了 `xActiveMask`，让 inactive token 不参与 routing。

参与者：AIV、GM、UB。

关键代码：`ApplyXActiveMask`，`dispatch_ffn_combine_kernel.hpp` 第 357-396 行。

输入：`expertIdx`、`xActiveMask`。

输出：被 mask 的 token 对应 expert id 改为 sentinel `expertNum`。

为什么这样做：

后续 `moe_init_routing_quant_v2` 会按 expert id routing。把 inactive token 的 expert id 改成 `expertNum`，相当于让它们落到无效 expert 或跳过路径。这里的具体 sentinel 处理逻辑依赖 `moe_init_routing_quant_v2`，需要进一步确认。

阶段 2: InitRouting + dynamic quant

目标：根据 `expertIdx` 对 token 做路由、排序/展开，并量化 token，写入 peermem。

参与者：AIV、GM、workspace、peermem。

关键代码：`DispatchAndCombine` 第 803-811 行调用 `moe_init_routing_quant_v2<ElementD2>`。

输入：

- 原始 `x`
- `expertIdx`
- `topK`
- InitRouting tiling
- peermem `offsetA`
- peermem `offsetPeerPerTokenScale`
- local `tokenPerExpert`

输出：

- `shmem() + peermemInfo.offsetA`：量化后的 routed token。
- `workspaceInfo.expandedRowIdx`：unpermute 用索引。
- `localTokenPerExpert`：本 rank 的 token count。
- `shmem() + peermemInfo.offsetPeerPerTokenScale`：per-token scale。

为什么这样做：

后续 GEMM 使用 int8 `AType`，因此 routing 时顺便 dynamic quant，把 token 和 scale 组织成适合 all-to-all / pull 的 peermem 布局。

阶段 3: 跨 rank token count 交换与 cumsum

目标：让每个 rank 知道每个远端 rank、每个 expert group 有多少 token，并计算本地 GEMM 需要的 M 维和 combine 写回偏移。

参与者：AIV、peermem、workspace、rank 间同步。

关键代码：

- `CrossRankSyncAndLocalTokenPerExpertAllGatherAndGetSumPreRankV2` 第 652-728 行。
- `GetCumsumForMMAIV` 第 399-429 行。
- `DispatchAndCombine` 第 815-840 行。

输入：local `tokenPerExpert`。

输出：

- 全 rank 可见的 `tokenPerExpert` 矩阵。
- `preSumBeforeRank`：combine 目标 offset。
- `cumsumMM`：GMM 分组 M 维。
- `expert_token_nums`：本 rank 每个 expert 的总 token 数。

为什么这样做：

GEMM 以本地 expert group 为单位做批处理，需要知道每个 group 总共有多少 token；combine 写回远端 peermem 时，需要知道这些 token 在目标 rank 的 expert 分段里应该写到哪里。

阶段 4: AIV 按 expert group 从远端 peermem Pull token

目标：把各 rank 发往本 rank expert 的量化 token 拉到本地 workspace `gmA`，并拉取 per-token scale 到 `gmPerTokenScale1`。

参与者：AIV、peermem、GM workspace、UB、MTE。

关键代码：

- `DispatchAndCombine` 第 851-909 行。
- `CopyGMToGMPerToken` 第 310-352 行。

输入：

- 远端 peermem `offsetA`
- `tokenPerExpert`
- `cumsumMM`
- `preSumBeforeRank`

输出：

- 本地 workspace `gmA`
- 本地 workspace `gmPerTokenScale1`

为什么这样做：

AIC 的 GEMM 需要本地连续的 `[tokens_for_expert, hidden]` 矩阵。各 rank 的 token 原本分散在远端 peermem 中，因此 AIV 多核按 `dstEpIdx` 分片拉取。每个 expert group pull 完后，通过 `CrossCoreSetFlag` 通知 AIC 可以开始该 group 的 GMM1，见第 885-888 行。

阶段 5: AIC GMM1

目标：对每个本地 expert group 执行第一层 FFN GEMM: $gmA * w1 \rightarrow gmC$ 。

参与者：AIC/Cube、workspace、权重 GM、scale GM、BlockMmad、BlockScheduler。

关键代码：`GMM1` 第 432-533 行。

输入：

- `gmA`
- `w1`
- `scale1`
- `cumsumMM`

输出：`gmC`。

为什么这样做：

每个本地 expert 的 token 数不同，所以按 `groupIdx` 动态读取 `currentM = cumsumMM((EP - 1) * expertPerRank + groupIdx)`。`BlockScheduler` 再把该 group 的 GEMM 切成 tile 分配给 AIC core。

GMM1 在开始时等待 AIV 完成 `cumsumMM`，并在每个 group 内等待 AIV pull 完该 group 的 token。见第 447-480 行。

阶段 6: AIV SwiGLU + 再量化

目标：对 GMM1 输出执行 per-token dequant、SwiGLU、再量化，生成 GMM2 输入。

参与者：AIV、workspace、UB、BlockEpilogue1。

关键代码：

- `DispatchAndCombine` 第 942-976 行。
- `block_epilogue_pertoken_swiglu.hpp` 第 145-282 行。

输入：

- `gmC`
- `gmPerTokenScale1`

输出：

- `gmPermutedToken`
- `gmPerTokenScale2`

为什么这样做：

GMM1 输出为 **N**，SwiGLU 将其拆成两半：一半做 sigmoid-like 激活，另一半相乘，输出维度约为 **N/2**，再量化为 int8 作为 GMM2 输入。这样可以让 GMM2 继续使用 int8 GEMM。

该阶段分两波处理：第一波在 **SYNCFLAGC2V** 后启动，完成后设置 **SYNCFLAGV2C** 让 AIC 开始 GMM2；若 **epilogueGranularity < expertPerRank**，第二波再重复一次。见第 942-973 行。

阶段 7：AIC GMM2

目标：对 SwiGLU 后的 token 执行第二层 FFN GEMM：**gmPermutedToken * w2 -> gmC2**。

参与者：AIC/Cube、workspace、权重 GM、scale GM。

关键代码：**GMM2** 第 536-633 行。

输入：

- **gmPermutedToken**
- **w2**
- **scale2**
- **cumsumMM**

输出：**gmC2**。

为什么这样做：

这是 FFN 第二层 projection，输出维度回到 **K**。GMM2 等待 AIV SwiGLU 完成对应分段后开始，见 **operator()<AIC>** 第 217-219 行和 **GMM2** 第 586-588 行。

阶段 8：CombineV1 / CombineV2 写回 peermem

目标：把 GMM2 输出按 rank/expert/token 布局写回目标 rank 的 peermem **offsetD**。

参与者：AIV、workspace **gmC2**、**gmPerTokenScale2**、peermem、BlockEpilogue2/3。

关键代码：

- **DispatchAndCombine** 第 977-986 行。
- **CombineV1** 第 1006-1047 行。
- **CombineV2** 第 1050-1109 行。
- **block_epilogue_pertoken_v2.hpp** 第 123-230 行。

输入：

- **gmC2**
- **gmPerTokenScale2**
- **tokenPerExpert**
- **preSumBeforeRank**
- **cumsumMM**

输出：远端 rank peermem **offsetD**。

为什么这样做：

GMM2 输出在本 rank 是按本地 expert group 聚合排列的，但最终每个 token 的结果应回到原始 token 所在 rank，并且还要按原 token/topK 顺序参与 unpermute。因此 combine 不能简单 memcpy，必须根据 **tokenPerExpert** 和 **preSumBeforeRank** 做分段 scatter。

阶段 9：CrossRankSync + Unpermute

目标：确保所有 rank 的 combine 写回完成，然后按 **expandedRowIndex** 和 **probs** 还原最终输出。

参与者：AIV、peermem、workspace、GM output。

关键代码：

- **ResetTokenPerExpert** 第 730-743 行。
- **shmem.CrossRankSync()** 第 992-993 行，对应 **hccL_shmem.hpp** 第 166-183 行。
- **KernelMoeTokenUnpermute** 第 998-1002 行。

输入：

- **peermem offsetD**
- **expandedRowIndex**
- **probs**

输出：**ptrOutput / out**。

为什么这样做：

combine 写回远端 peermem 是跨 rank 的；若不做 rank 间同步，本 rank 可能在远端结果尚未全部写入时开始 unpermute，导致读到旧数据或未完成数据。
`expandedRowIdx` 记录了 routing 后 token 与原 token/topK 的关系，`probs` 用于 topK 加权。

4. AIC 计算路径简要说明

AIC/Cube 在该算子中主要负责两次 GEMM：

- GMM1: `gmA * w1 -> gmC`，见 `dispatch_ffn_combine_kernel.hpp` 第 432-533 行。
- GMM2: `gmPermutedToken * w2 -> gmC2`，见第 536-633 行。

`BlockMmad` 是本算子使用的 GEMM block 执行器，类型在 `dispatch_ffn_combine.h` 第 225 行定义，具体实现见 `utils/block_mmad_preload_async_fixpipe_quant.hpp`。其职责包括：

- 从 GM 把 A/B/scale tile 搬到 L1/L0。
- 使用 MMAD 执行矩阵乘。
- 对 int8 GEMM 使用 `fixpipe / scale` 相关写回逻辑。
- 通过 `SynchronizeBlock()` 排空异步 preload pipeline。
- 通过 `Finalize()` 设置 cross-core flag，通知 AIV 某些 group 的 GMM 已完成。

GEMM 按 expert group 切分：外层循环 `groupIdx` 遍历 `expertPerRank`。每个 group 的 M 维来自 `cumsumMM((EP - 1) * expertPerRank + groupIdx)`，即所有 rank 发到本 rank 该 expert 的 token 总数。

每个 group 内，`BlockScheduler` 根据 `L1TileShape: M/N` 把 GEMM 切成 block，并把 block 分配给 AIC core。关键调用包括：

- `blockScheduler.Update(...)`
- `GetCoreLoops()`
- `GetBlockCoord(loopIdx)`
- `GetActualBlockShape(blockCoord)`

结果写入：

- GMM1 写 `gmC`，供 AIV SwiGLU。
- GMM2 写 `gmC2`，供 AIV Combine。

5. AIV 向量与搬运路径简要说明

AIV/Vector 在该算子中承担除 Cube GEMM 外的大部分工作：

- `ApplyXActiveMask`：处理可选 active mask。
- `moe_init_routing_quant_v2`：routing、排序/展开、dynamic quant。
- `CrossRankSyncAndLocalTokenPerExpertAllGatherAndGetSumPreRankV2`：跨 rank token count 交换与 prefix sum。
- `CopyGMToGMPerToken`：从远端 peermem pull token 与 per-token scale。
- `BlockEpilogue1`：GMM1 输出 dequant、SwiGLU、quant。
- `BlockEpilogue2/3`：GMM2 输出 dequant、cast，并 scatter 到远端 peermem。
- `KernelMoeTokenUnpermute`：按 `expandedRowIdx` 和 `probs` 还原最终输出。

AIV 会使用：

- `expandedRowIdx`：最终 unpermute 索引。
- `perTokenScale1`：GMM1 输出反量化。
- `perTokenScale2`：GMM2 输出 combine 反量化。
- `tokenPerExpert`：dispatch pull 和 combine scatter 的 rank/expert/token 分段依据。
- `preSumBeforeRank`：combine 写回目标 rank peermem 的 offset。

UB 主要作为 AIV 的搬运和向量计算 scratchpad：

- `CopyGMToGM / CopyGMToGMPerToken` 使用 UB ping-pong 搬运 GM/peermem 数据。
- `BlockEpilogue1` 在 UB 中做 cast、Muls、Exp、Div、Mul、ReduceMax、quant。
- `BlockEpilogue2/3` 在 UB 中做 cast、per-token scale 乘法、cast back、copy out。

AIV 和 AIC 存在明显分工协作：AIV 生产 GEMM 输入、消费 GEMM 输出；AIC 消费 AIV 准备好的连续 token buffer，生产 `gmC/gmC2`。双方通过 `CrossCoreSetFlag / CrossCoreWaitFlag` 协调。

6. 通信与计算的调度关系

6.1 是否存在 MC2 / all-to-all / rank 间通信

存在。证据包括：

- `dispatch_ffn_combine_def.cpp` 第 93 行注册 `.MC2().HcclGroup("group")`。
- `dispatch_ffn_combine_tiling.cpp` 第 310-317 行使用 `Mc2CcTilingConfig`，配置 `AlltoAll=level0:fullmesh;level1:pairwise`。
- `HcclShmem` 第 84-183 行读取 HCCL context，并通过 HCCL window 获取本地/远端 peermem。
- `DispatchAndCombine` 第 871-881 行从 `shmem(0, dstEpIdx)` 读取远端 token。
- `BlockEpilogueV2` 第 217-225 行通过 `params.shmem(params.offsetD, dstEpIdx)` 写远端 peermem。

这不是调用一个显式 `AllToAll` API，而是通过 MC2/HCCL window 暴露的 `peermem` 做远端读写，语义上实现了 all-to-all token exchange 和 result exchange。

6.2 通信发生在哪些阶段

- InitRouting 阶段：本 rank 把 routed token 和 scale 写到本地 `peermem`。
- token count exchange 阶段：`CrossRankSyncAndLocalTokenPerExpertAllGatherAndGetSumPreRankV2` 把本 rank token count 写到远端 `peermem`，并等待远端写入。
- Dispatch Pull 阶段：AIV 从远端 rank `peermem offsetA` 拉取发往本 rank expert 的 token。
- Combine 阶段：AIV 把本 rank expert 计算完成的结果写回目标 rank `peermem offsetD`。
- Unpermute 前：`CrossRankSync()` 保证所有 rank combine 写回完成。

6.3 通信前后数据布局变化

通信前：

- `x` 是按本 rank 原始 token 顺序排列。
- `expertIdx` 表示每个 token 的目标 expert。

InitRouting 后：

- token 按 expert/rank 分组，量化后写到 `peermem offsetA`。
- `expandedRowIdx` 记录原顺序映射。
- `tokenPerExpert` 记录每个 rank/expert 的 token 数。

Dispatch Pull 后：

- 本 rank workspace `gmA` 中 token 按本地 expert group 连续排列。
- `cumsumMM` 提供每组起点/长度。

Combine 后：

- GMM2 输出从本地 expert-group layout scatter 到目标 rank `peermem offsetD`。
- `peermem offsetD` 中的数据布局适合 `KernelMoeTokenUnpermute` 根据 `expandedRowIdx` 读回。

6.4 是否存在通信-计算重叠

存在一定程度的重叠，尤其是 AIV Pull 与 AIC GMM1 之间。

证据：

- AIV 每完成一个 expert group 的 Pull，就 `CrossCoreSetFlag` 通知 AIC，见 `DispatchAndCombine` 第 885-888 行。
- AIC 的 `GMM1` 在每个 group 计算前等待对应 flag，见第 477-480 行。
- 这意味着 AIV 可以继续 pull 后续 group，AIC 同时计算已准备好的前面 group，形成 producer-consumer。

GMM1 和 SwiGLU / GMM2 也有流水：

- GMM1 在 `epilogueGranularity` 位置调用 `Finalize(..., SYNCFLAGC2V)`，见第 505-512 行。
- AIV 等 `SYNCFLAGC2V` 后做第一波 SwiGLU，完成后 `SetFlag(SYNCFLAGV2C)`，见第 942-956 行。
- AIC 等 `SYNCFLAGV2C` 后进入 GMM2，见第 217-219 行和第 586-588 行。

Combine 与 GMM2：

- CombineV1 会按 group 等待 AIC flag，见第 1011-1014 行。
- CombineV2 在每个 group 里等待 `CrossCoreWaitFlag`，见第 1089-1092 行。
- 说明 combine 消费 GMM2 输出，也按 group/tile 形成同步关系。

6.5 生产者-消费者关系

- AIV InitRouting / Pull 是 AIC GMM1 的生产者。
- AIC GMM1 是 AIV SwiGLU 的生产者。
- AIV SwiGLU 是 AIC GMM2 的生产者。
- AIC GMM2 是 AIV Combine 的生产者。
- AIV Combine 是最终 Unpermute 的生产者。

6.6 flag、sync、workspace、pipe、双缓冲

主要协调机制：

- `AscendC::SyncAll<true>()`：AIV/AIC 内核内全 core 同步，保护共享 workspace 或 `peermem` 状态。
- `CrossCoreSetFlag<0x2>` / `CrossCoreWaitFlag<0x2>`：AIC/AIV 或核间阶段同步。
- `HardEvent` flag：UB/MTE/Vector/FIX pipe 之间同步，例如 `MTE2_V`、`V_MTE3`、`MTE3_MTE2`。
- `gm_signal_wait_until_ne`：轮询远端 `peermem` 中的 token count 信号。
- `gm_dcci`：cache clean/invalidate，确保远端写入可见。
- UB ping-pong：`CopyGMToGM`、`CopyGMToGMPerToken`、`BlockEpilogue2/3` 都使用双 buffer 或多 event。

- **BlockMmad** preload stages: **PRELOAD_STAGES=1**、**L1_STAGES=2**、**L0A/L0B_STAGES=2**，在 **block_mmad_preload_async_fixpipe_quant.hpp** 第 190-263 行体现 GM->L1 preload 与 MMAD 交叠。

6.7 关键同步点

- InitRouting 后 **SyncAll**: 保证 **expandedRowIdx/tokenPerExpert/peermem token** 写入完成。
- token count exchange 后 **SyncAll**: 保证全 rank token count 和 **preSumBeforeRank** 可用。
- 每 group Pull 后 **CrossCoreSetFlag**: 保证 AIC 不会读未 pull 完的 **gmA**。
- **SYNCFLAGC2V**: 保证 AIV SwiGLU 不会读未完成的 **gmC**。
- **SYNCFLAGV2C**: 保证 AIC GMM2 不会读未完成的 **gmPermutedToken**。
- Combine 前 group flag: 保证 AIV Combine 不会读未完成的 **gmC2**。
- Combine 后 **CrossRankSync()**: 保证 unpermute 不会读未写完的远端 **peermem**。

7. BlockEpilogue / Combine 阶段详解

7.1 CombineV2 的输入

CombineV2 定义在 **dispatch_ffn_combine_kernel.hpp** 第 1050-1109 行，输入包括：

- **gmC2**: GMM2 输出，按本 rank expert group 连续排列。
- **gmPerTokenScale2**: SwiGLU 再量化时生成的 scale，用于 GMM2 输出反量化。
- **groupId**: 当前本地 expert group。
- **preSrcExpertSum**: 当前 group 在 **gmC2** 中的起始行。
- **preSumBeforeRank**: 写回目标 rank **peermem** 的行偏移。
- **tokenPerExpert**: 每个目标 rank 在当前 expert group 的 token 数。
- **BlockScheduler** 输出的 **blockCoord / actualBlockShape**: 当前 GEMM tile。

BlockEpilogue3::Params 在 **DispatchAndCombine** 第 923-934 行构造，传入：

- **EP**
- **expertPerRank**
- **rank**
- **ptrTokenPerExpert**
- **layoutD2**
- **n2**
- **L1TileShape::N**
- **shmem**
- **offsetD**
- **tokenPerExpertLayout**

7.2 CombineV2 如何把结果合并回去

CombineV2 的核心是按 expert group 和 GEMM tile 遍历 **gmC2**：

- 第 1061-1068 行得到当前 group 的 **currentExpertM**。
- 第 1068-1075 行用 **BlockScheduler** 得到 tile。
- 第 1076-1086 行把 tile 的 M 维进一步按 **m0 = 16** 切小块，并根据 **get_subblockid()** 分给两个 AIV subblock。
- 第 1089-1092 行等待 AIC 对应 group/tile 的 GMM2 完成。
- 第 1094-1102 行对每个小 M 块调用 **blockEpilogue(...)**。

BlockEpilogue3::operator() 位于 **block_epilogue_pertoken_v2.hpp** 第 123-230 行。它做了几件事：

1. 根据 **preSrcExpertSum + blockCoord.m()** 和 **blockCoord.n()** 定位 **gmC2** tile，见第 138-144 行。
2. 把 **gmC2** tile 从 GM copy 到 UB，cast 成 fp32，见第 147-153 行。
3. 读取 **gmPerTokenScale2**，每行乘对应 scale，完成 per-token dequant，见第 159-179 行。
4. cast 到输出 dtype **ElementD**，见第 181-185 行。
5. 遍历 **dstEpIdx**，通过 **tokenPerExpert** 判断当前 tile 的哪些行属于哪个目标 rank，见第 193-211 行。
6. 通过 **preSumBeforeRank(dstEpIdx * expertPerRank + groupId)** 得到目标 rank **peermem** 内的写入起点，见第 195-221 行。
7. 把 UB 中对应子段 copy 到 **params.shmem(params.offsetD, dstEpIdx)**，见第 217-225 行。

7.3 per-token scale 是否在 CombineV2 使用

使用。**BlockEpilogue3::operator()** 第 159-164 行从 **gmPerTokenScale** 读取 scale；第 176-179 行对每行 **ubCFp32** 乘 scale。该 scale 是 **gmPerTokenScale2**，由 SwiGLU 再量化阶段写出，见 **block_epilogue_pertoken_swiglu.hpp** 第 275-282 行。

7.4 关键变量含义

- **cur_row**: **CombineV2** 中当前 AIV subblock 负责的小 M 块编号，见第 1094 行。
- **aiv_m_rows**: 当前 AIV subblock 需要处理多少个 **m0=16** 的小块，见第 1078-1082 行。
- **preSrcExpertSum**: 当前 expert group 在 **gmC2** 中的累计起始行。每处理完一个 group，加上 **currentExpertM**，见第 1105 行。
- **preSumBeforeRank**: 目标 rank **peermem** 中当前 expert group 的写入基准行，见 **BlockEpilogue3** 第 196、220 行。

- `blockCoord.n()`: 当前 tile 在 N 维的起始列。报告要求中的 `n_offset` 在当前代码中没有同名变量, 最近的是 `blockCoord.n() * L1TileShape::N` 和 `blockCoord.n()`; 它表示当前 combine tile 写回时的列偏移。
- `stTile/edTile`: 当前 tile 在当前 expert group 内覆盖的源行范围, 见 `block_epilogue_pertoken_v2.hpp` 第 187-189 行。
- `stRankInExpert/edRankInExpert`: 某个目标 rank 在当前 expert group 内的源行范围, 见第 197-199 行。
- `dstOffsetInExpert`: 如果 tile 从某个 rank 的分段中间开始, 写回该 rank peermem 时需要加的 rank 内偏移, 见第 213-216 行。
- `tileOffset`: UB tile 中已经写出的行数偏移, 见第 191、225-226 行。

7.5 Finalize() 做什么

`BlockEpilogue3::Finalize()` 位于 `block_epilogue_pertoken_v2.hpp` 第 104-116 行。它等待此前 `SetFlag()` 初始化/使用过的 hard event:

- `V_MTE2`
- `S_MTE2`
- `MTE3_V`

本质上是确保该 epilogue 内部所有 GM->UB、Vector、UB->GM 的 pipeline 操作完成, 避免 kernel 后续阶段复用 UB 或进入 `CrossRankSync` 时仍有未完成写回。

`CombineV2` 在所有 group 结束后调用 `blockEpilogue.Finalize()`, 见 `dispatch_ffn_combine_kernel.hpp` 第 1108 行。

7.6 为什么 combine 不能简单 memcpy

原因有三点:

1. `gmC2` 是按本 rank 的本地 expert group 聚合排列的, 不是按原始 token 顺序排列。
2. 每个 group 内的 token 又按来源/目标 rank 分段, 不同 rank 的分段长度由 `tokenPerExpert` 决定。
3. 最终输出需要回到 token 原 rank, 并按 `expandedRowIndex` 和 `probs` 做 topK 加权合并。

因此 combine 必须根据 `tokenPerExpert` 找到当前 tile 覆盖了哪些 rank 的 token, 再根据 `preSumBeforeRank` 写到目标 rank peermem 中正确位置。

7.7 小例子

假设:

- 2 个 rank: rank0、rank1。
- 每个 rank 1 个 expert: expert0 在 rank0, expert1 在 rank1。
- 4 个 token: `t0, t1` 初始在 rank0, `t2, t3` 初始在 rank1。
- 路由结果: `t0->expert1, t1->expert0, t2->expert0, t3->expert1`。

Dispatch 后:

- rank0 本地 peermem `offsetA` 中有 `t0` 发给 rank1/expert1, `t1` 留给 rank0/expert0。
- rank1 本地 peermem `offsetA` 中有 `t2` 发给 rank0/expert0, `t3` 留给 rank1/expert1。
- `tokenPerExpert` 记录 rank0/rank1 到各目标 expert 的 token 数。

rank0 Pull:

- rank0 从自己的 peermem 拿 `t1`, 从 rank1 peermem 拿 `t2`, 组成 expert0 的连续 `gmA=[t1, t2]`。

rank1 Pull:

- rank1 从 rank0 peermem 拿 `t0`, 从自己的 peermem 拿 `t3`, 组成 expert1 的连续 `gmA=[t0, t3]`。

FFN:

- rank0 AIC 对 `[t1, t2]` 跑 expert0 FFN, 得到 `y1, y2`。
- rank1 AIC 对 `[t0, t3]` 跑 expert1 FFN, 得到 `y0, y3`。

Combine:

- rank0 计算出的 `y1` 应写回 rank0, `y2` 应写回 rank1。
- rank1 计算出的 `y0` 应写回 rank0, `y3` 应写回 rank1。
- 写回位置由 `preSumBeforeRank` 和 `expandedRowIndex` 间接保证。

Unpermute:

- rank0 从自己的 peermem `offsetD` 读回 `y0, y1`, 按原 token/topK 和 `probs` 写 `out`。
- rank1 读回 `y2, y3`, 写自己的 `out`。

8. 关键同步与正确性保障

8.1 `AscendC::SyncAll<true>()` after InitRouting

位置: `DispatchAndCombine` 第 813 行。

保护资源: `expandedRowIndex`、local `tokenPerExpert`、peermem `offsetA`、per-token scale。

去掉风险：后续 token count exchange 或 pull 可能读到未完成的 routing/quant 数据。

类型：核内同步。

性能影响：会形成阶段边界，但 routing 输出是后续通信和 GEMM 的必要输入。

8.2 token count exchange 中的远端信号等待

位置：CrossRankSyncAndLocalTokenPerExpertAllGatherAndGetSumPreRankV2 第 652-728 行，尤其第 690-696 行 gm_signal_wait_until_ne。

保护资源：远端 rank 写入的 tokenPerExpert。

去掉风险：读取未写完或旧的 token count，导致 cumsum/GEMM M 维/combine offset 全错。

类型：通信同步。

性能影响：可能受 rank 数、window 访问延迟、负载不均影响。

8.3 GetCumsumForMMAIV 后 SyncAll

位置：DispatchAndCombine 第 821-832 行。

保护资源：cumsumMM、preSumBeforeRank。

去掉风险：AIC 可能基于未完成的 cumsum 计算错误 M 维；AIV pull 可能用错行偏移。

类型：核内同步。

性能影响：小，但在所有 group 计算前是硬依赖。

8.4 AIV Pull 完 group 后通知 AIC

位置：AIV 第 885-888 行；AIC GMM1 第 477-480 行。

保护资源：gmA 和 gmPerTokenScale1 中当前 expert group 的数据。

去掉风险：AIC GMM1 读取尚未从远端 peermem pull 完的 token。

类型：AIC-AIV 同步。

性能影响：这是通信-计算流水的关键。同步太粗会降低重叠，太细会增加 flag 开销。

8.5 SYNCFLAGC2V

位置：常量第 60 行；GMM1 Finalize(..., SYNCFLAGC2V) 第 505-512、528-532 行；AIV wait 第 942-944、960-961 行。

保护资源：gmC。

去掉风险：AIV BlockEpilogue1 读取未完成的 GMM1 输出。

类型：AIC-AIV 同步。

性能影响：控制 GMM1 与 SwiGLU 流水粒度。

8.6 SYNCFLAGV2C

位置：常量第 61 行；AIV set 第 954-956、972-973 行；AIC wait 第 217-219、586-588 行。

保护资源：gmPermutedToken 和 gmPerTokenScale2。

去掉风险：AIC GMM2 读取未完成的 SwiGLU/quant 输出。

类型：AIC-AIV 同步。

性能影响：决定 GMM2 能否尽早启动。

8.7 BlockMmad 内部 hard event

位置：block_mmad_preload_async_fixpipe_quant.hpp 第 190-294 行。

保护资源：L1/L0A/L0B/LOC、MTE/FIX/MMAD pipeline。

去掉风险：tile 搬运、MMAD、写回乱序导致数据覆盖或读未完成 tile。

类型：pipe 内同步。

性能影响：必要的 pipeline 正确性开销；设计目标是用 preload stages 隐藏搬运延迟。

8.8 BlockEpilogue 内部 hard event

位置：

- `block_epilogue_pertoken_swiglu.hpp` 第 203-282 行。
- `block_epilogue_pertoken_row.hpp` 第 158-187 行。
- `block_epilogue_pertoken_v2.hpp` 第 147-185、193-228 行。

保护资源：UB buffer、GM->UB copy、Vector 计算、UB->GM copy。

去掉风险：UB 被覆盖、scale 未读完、cast 与 copy out 顺序错误。

类型：AIV pipe 内同步。

性能影响：局部 pipeline 开销；通过双 buffer / ping-pong 降低等待。

8.9 Combine 等待 GMM2 完成

位置：

- CombineV1 第 1011-1014 行。
- CombineV2 第 1089-1092 行。

保护资源：`gmC2`。

去掉风险：combine 读取未完成的 GMM2 输出。

类型：AIC-AIV 同步。

性能影响：决定 GMM2 与 combine 是否可以 group/tile 粒度流水。

8.10 ResetTokenPerExpert

位置：`DispatchAndCombine` 第 988-990 行；函数定义第 730-743 行。

保护资源：peermem 中 `tokenPerExpert` 区。

去掉风险：下一次 kernel 调用可能读到旧 token count。

类型：状态清理同步，函数内部只由最后一个 AIV core 执行清零。

性能影响：与 $EP * paddedExpertNumAligned$ 大小相关。

8.11 shmem.CrossRankSync()

位置：`DispatchAndCombine` 第 992-993 行；`hccl_shmem.hpp` 第 166-183 行。

保护资源：所有 rank 的 peermem `offsetD` combine 输出。

去掉风险：unpermute 读取远端尚未写完的数据。

类型：通信同步 / rank 间 barrier。

性能影响：可能是尾部延迟瓶颈，受最慢 rank 影响。

9. 性能设计意图

9.1 融合减少 GM 读写和 launch

普通 MoE pipeline 可能包含独立的 routing、all-to-all、GEMM1、activation、GEMM2、combine、unpermute 多个算子。该实现把它们融合在一个 kernel 内：

- routing 直接写 peermem 和 workspace。
- GMM1 输出 `gmC` 直接被 AIV epilogue 消费。
- SwiGLU 输出 `gmPermutedToken` 直接被 GMM2 消费。
- GMM2 输出 `gmC2` 直接被 combine epilogue 消费。
- combine 直接写 peermem，随后 unpermute 写最终 `out`。

这减少了跨 kernel 的中间 GM 读写、kernel launch 和全局调度开销。

9.2 AIC / AIV 分工提高并行度

AIC 负责高吞吐 GEMM；AIV 负责不适合 Cube 的工作：

- routing / sorting / quant
- peermem copy
- scale/cast/SwiGLU
- scatter combine
- unpermute

这种分工类似 CUDA 中 tensor core kernel 与独立 copy/epilogue 逻辑融合，但 Ascend 中通过 AIC/AIV 混合 task 和 cross-core flag 做显式协作。

9.3 通信与计算重叠

最明显的重叠是：

- AIV 按 group Pull token，pull 完一个 group 立刻通知 AIC 做 GMM1。
- AIV 可继续 pull 后续 group，AIC 同时计算前面 group。

其次是：

- GMM1 分两波通知 AIV 做 SwiGLU。
- AIV SwiGLU 完第一波后通知 AIC GMM2。
- CombineV1/V2 按 group/tile 等 GMM2 flag 消费输出。

因此该 kernel 不是严格的“全部通信完成后计算”，而是有多处 producer-consumer 重叠。

9.4 按 expert / group / block 调度的好处

- expert group 是 MoE 的自然批处理单位，可避免把不同 expert 权重混在同一个 GEMM。
- `cumsumMM` 提供每个 group 的真实 M，适配 token 分布不均。
- `BlockScheduler` 让每个 group 内 GEMM 按 `128x256x512` L1 tile 分配给多个 AIC core。
- `startCoreIdx` 在 group 之间滚动，减少 group 间负载不均导致的 core 利用率问题。
- CombineV2 对小 batch 使用 tile 级 scatter，避免 CombineV1 按整 group 等待造成过粗同步。

9.5 Workspace / UB 复用策略

workspace 线性切分，复用同一大块 GM：

- routing index / cumsum / scale / GMM 中间结果 / soft flags 都在 `ptrWorkspace` 内。
- peermem 切分为 token 输入、scale、结果、token count 区。

UB 用于：

- GM/peermem copy 的 ping-pong buffer。
- epilogue 的 fp32 临时 buffer。
- scale buffer。
- quant / cast 临时 buffer。

`BlockEpilogue3` 中 `ubCList/ubDList/ubFp32List/scaleUbList` 双 buffer 加 event，减少 MTE 与 Vector 串行等待。

9.6 可能的性能瓶颈

- **token 分布不均**：某些 expert 收到大量 token，某些 group 很小，会导致 AIC core 利用率下降或尾部 rank 等待。
- **rank 数增加**：`tokenPerExpert` 交换、peermem 远端访问、`CrossRankSync` 成本随 EP 上升。
- **topK 增大**：routing token 数变为 $M * \text{topK}$ ，`expandedRowIdx`、peermem token、combine/unpermute 压力增大。
- **hidden size K / FFN N**：直接影响 GMM1/GMM2 计算量、workspace 大小和 peermem 搬运量。
- **maxOutputSize**：决定 workspace 大小和最多处理 token 数，过大增加内存占用，过小会截断或限制吞吐。
- **HCCL_BUFFER_SIZE/window size**：host tiling 第 281-287 行会检查 $M * \text{topK} * K * \text{sizeof}(\text{int8_t}) * 3 + 10\text{MB}$ 是否超过 window。
- **Combine scatter**：不是连续 memcpy，而是按 rank/expert 分段 scatter，尤其小 batch 或 token 分布碎片化时开销明显。
- **同步粒度**：`CrossCoreSetFlag/WaitFlag` 太频繁会增加开销；太粗会降低通信-计算重叠。当前通过 `epilogueGranularity` 和 CombineV2 tile 化在两者之间折中。

9.7 对性能敏感的参数

- **M**：本 rank token 数。
- **topK**：expanded token 数倍增因子。
- **K**：hidden size，也是 GMM2 输出维度。
- **N**：FFN 第一层输出维度，SwiGLU 后为 $N/2$ 。
- **expertPerRank**：本 rank expert group 数，影响 group 循环、sync 次数和负载均衡。
- **EP/worldSize/rankSize**：跨 rank peermem 访问和同步规模。
- **maxOutputSize**：workspace 与截断边界。
- **L1TileShape = 128x256x512、L0TileShape = 128x256x128**：决定 GEMM tile 粒度。
- **epilogueGranularity**：决定 GMM1/SwiGLU/GMM2 流水切分。
- **ubMoveNum**：GM/UB 搬运分块大小。

10. 参考：dispatch_ffn_combine AIC/AIV 执行时间线

改 mc2 kernel 时，需理解 AIC (Cube) 与 AIV (Vector) 并行、通过 CrossCore flag 握手的 producer-consumer 流水线。

10.1 核类型与入口分流

910B 上 1 个 AI Core = 1 AIC + 2 AIV（硬件固定）；算子通过 `KERNEL_TYPE_MIX_AIC_1_2` 显式声明 1:2 混合调度：

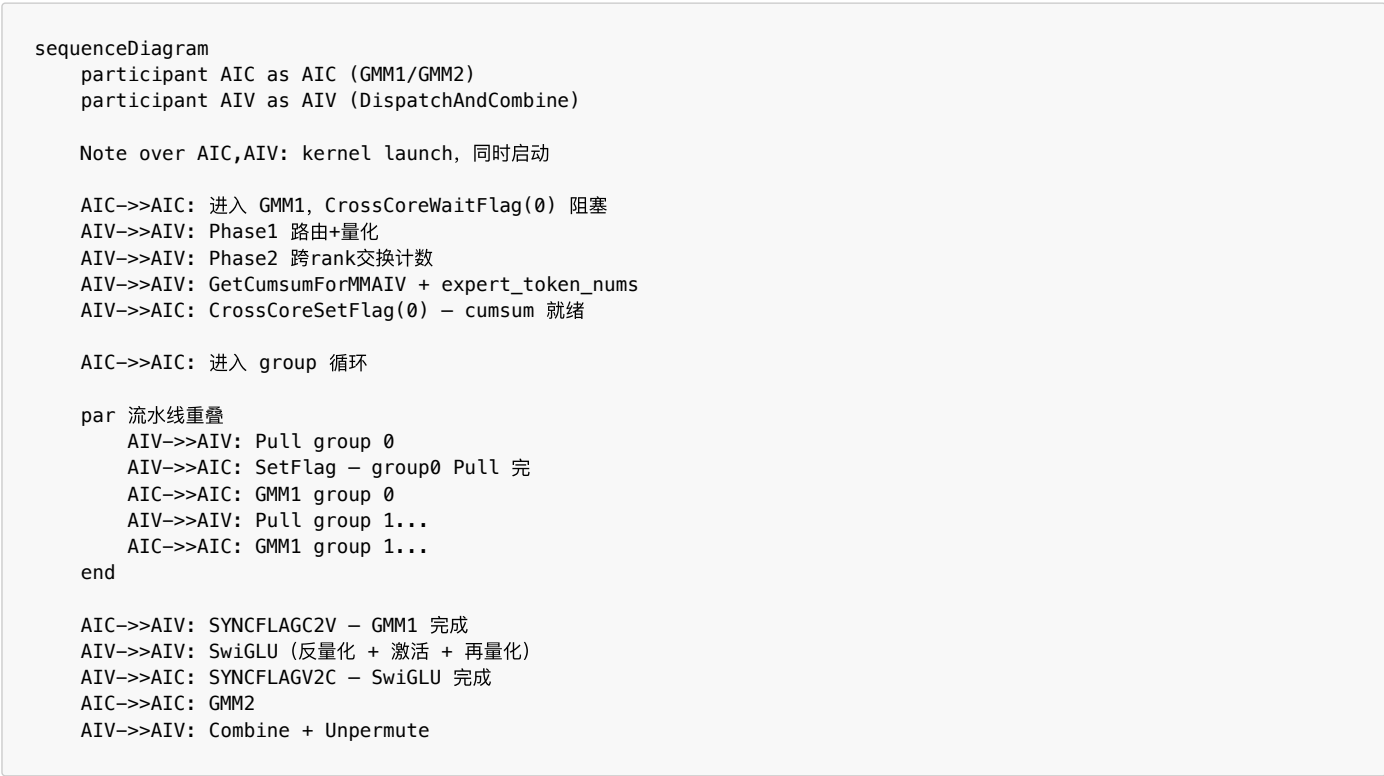
```
// csrc/mc2/dispatch_ffn_combine/op_kernel/dispatch_ffn_combine.cpp
KERNEL_TASK_TYPE(1000010, KERNEL_TYPE_MIX_AIC_1_2);
```

Process() 内所有 core 都调用 kernel(params)，但按 core 类型走不同路径 (dispatch_ffn_combine_kernel.hpp)：

Core	入口	职责
AIC	operator()<AIC> → GMM1 → GMM2	两次 expert FFN GEMM
AIV	operator()<AIV> → DispatchAndCombine	路由、跨卡同步、Pull、SwiGLU、Combine、Unpermute

kernel launch 后 AIC 与 AIV 同时启动，不是串行先后关系。

10.2 时间线 (producer-consumer)



10.3 各阶段与代码位置

阶段	执行核	主要函数 / 同步点	源码位置 (约)
Phase 1 路由+量化	AIV	moe_init_routing_quant_v2	dispatch_ffn_combine_kernel.hpp 900-908
Phase 2 跨 rank 交换 token 计数	AIV	CrossRankSyncAndLocalTokenPerExpertAllGatherAndGetSumPreRankV2	910-913
cumsum + 输出 expert_token_nums	AIV (core 0)	GetCumsumForMMAIV	915-938
cumsum 就绪通知	AIV → AIC	CrossCoreSetFlag(0) / CrossCoreWaitFlag(0)	AIV 940-943; AIC 481-484
Phase 3 Pull token	AIV	每组 Pull 完 CrossCoreSetFlag(syncgmmIdx++)	949-986
GMM1	AIC	每组前 CrossCoreWaitFlag(syncgmmIdx); 结束 SYNCFLAGC2V	467-568
SwiGLU	AIV	等 SYNCFLAGC2V; 完 SYNCFLAGV2C	1040-1072
GMM2	AIC	等 SYNCFLAGV2C	218-219; 570+
Combine + Unpermute	AIV	CombineV1/V2 + KernelMoeTokenUnpermute	1075-1100

10.4 关键同步语义

Flag	方向	含义
CrossCoreSetFlag(0)	AIV → AIC	cumsumMM 已就绪，AIC 可读每组 M 维
每组 Pull 后的 SetFlag	AIV → AIC	该 expert 组 token 已从 peermem 拉到本地 gmA
SYNCFLAGC2V	AIC → AIV	GMM1 输出 gmC 就绪，AIV 可 SwiGLU
SYNCFLAGV2C	AIV → AIC	SwiGLU 输出 gmPermutedToken 就绪，AIC 可 GMM2

更完整的算子分析见：[csrc/mc2/dispatch_ffn_combine/dispatch_ffn_combine_kernel_report.md](#)。

结论

`dispatch_ffn_combine_kernel` 的核心不是单个 GEMM，而是一个完整的 Expert Parallel MoE 子图融合：AIV 负责 routing、通信搬运、epilogue、combine 和 unpermute；AIC 负责两次 expert FFN GEMM。它通过 workspace、peermem、`tokenPerExpert`、`cumsumMM`、`preSumBeforeRank` 和一系列 cross-core / cross-rank 同步，把 dispatch、FFN、combine 串成可流水的 producer-consumer 执行链。

需要进一步确认的点：

- `moe_init_routing_quant_v2` 内部对 sentinel expert id、`expandedRowIdx` 精确格式、`tokenPerExpert` 写入维度的具体定义。
- `WorkspaceInfo` 中第二段未命名 `EP * EP * expertPerRank` 空洞的真实用途。
- `ptrSoftFlagBase`、`InitArithProgress`、`UpdateAicFlags` 是否属于旧路径或特定编译配置下启用路径。
- CombineV1/CombineV2 的启发式阈值 `M * topK <= 4096` 是否经过特定模型/芯片调参。